📘 BÀI TẬP .NET 06 — CybersoftMarketPlace

Thực hành kiến trúc 4 tầng · Repository & Unit of Work Pattern

> **Kiến trúc bắt buộc:** `Controller → Service → UnitOfWork → Repository → DbContext`
> **Quy tắc vàng:** Controller không chứa business logic · không đụng DbContext trực tiếp · API chỉ trả **DTO** bọc trong `HTTPResponseData<T>`, không trả entity.
> **Phạm vi:** Cả 2 bài làm trên entity `Category` — đã có sẵn `CategoryRepository`, đã đăng ký DI và nằm trong `UnitOfWork` (bạn **không cần** tạo lại repository).

Entity `Category` có sẵn: `Id (int)`, `Name (string)`, `Alias (string?)`, `AdditionalData (string?)`, `Deleted (bool?)`, `ShopId (int)`, navigation `Shop`, `Products`.

🟢 Bài 1 — Lấy danh sách Category (GET)

Mục tiêu: *Làm quen luồng đọc dữ liệu và trả về DTO có phân trang.*
Mẫu tham khảo: *`ProductController.cs`* → *`ProductService.cs`*

Yêu cầu chức năng

Xây dựng endpoint `GET /api/Category/GetAll` trả về danh sách category, với:

- Tham số: `keyword` (tìm theo tên, mặc định rỗng), `pageIndex` (mặc định 1), `pageSize` (mặc định 10).
- Chỉ lấy các category **chưa bị xoá** (`Deleted == false`).
- Kết quả bọc trong `HTTPResponseData<List<CategoryDTO>>`.

Các thành phần phải tạo

Tầng	File cần tạo	Nội dung
Application/DTO	`CategoryDTO.cs`	Chỉ chứa trường cần trả: `Id`, `Name`, `Alias` (KHÔNG trả nguyên entity)
Application/Services	`CategoryService.cs`	`ICategoryService` + `CategoryService`, method `GetAllCategoriesAsync(...)`
Api/Controllers	`CategoryController.cs`	Inject `ICategoryService`, action `GetAll`
Api/Program.cs	(thêm 1 dòng DI)	`AddScoped<ICategoryService, CategoryService>()`

Ràng buộc & gợi ý

- Trong Service lấy repository qua `_unitOfWork.CategoryRepository`, dùng method `Where(...)` có sẵn của `IRepositoryBase`.
- Phân trang bằng `.Skip((pageIndex - 1) * pageSize).Take(pageSize)`.
- Map entity → `CategoryDTO` bằng `.Select(...)` (giống ProductService).
- Action phải có `/// <summary>` để Swagger đọc được.

✅ Tiêu chí hoàn thành

- [ ] Build không lỗi, không warning nullable mới.
- [ ] Swagger hiện endpoint `GET /api/Category/GetAll`, gọi thử trả JSON đúng cấu trúc `HTTPResponseData`.
- [ ] Controller **không** chứa LINQ/DbContext; Service **không** đụng DbContext trực tiếp.
- [ ] Response trả `CategoryDTO`, không lộ entity `Category`.

🟡 Bài 2 — Thêm mới Category (POST)

Mục tiêu: *Làm quen luồng ghi dữ liệu, validate input bằng DataAnnotation, xử lý trùng lặp.*
Mẫu tham khảo: `UserController.cs` → `UserService.cs`

Yêu cầu chức năng

Xây dựng endpoint `POST /api/Category/Create` để thêm 1 category mới:

- Nhận vào `CategoryCreateDTO` gồm: `Name` (bắt buộc, 2–100 ký tự), `ShopId` (bắt buộc).
- Kiểm tra **trùng tên trong cùng Shop** → nếu trùng trả về lỗi 400.
- Khi tạo: tự sinh `Alias` từ `Name` (dùng `HelperFunction.StringToSlug`), set `Deleted = false`.
- Trả về `HTTPResponseData<string>` với thông báo thành công/thất bại (giống `RegisterUserAsync`).

Các thành phần phải tạo / cập nhật

Tầng	File	Nội dung
Application/DTO	`CategoryCreateDTO.cs`	Có `[Required]`, `[StringLength]` cho `Name`; `[Required]` cho `ShopId`
Application/DTO	(bổ sung message)	Thêm class hằng thông báo, ví dụ `CategoryResponseMessageDTO` (mẫu: `UserResponseMessageDTO`)
Application/Services	`CategoryService.cs`	Thêm method `CreateCategoryAsync(CategoryCreateDTO model)`
Api/Controllers	`CategoryController.cs`	Thêm action `Create` (`[HttpPost("Create")]`)

Ràng buộc & gợi ý

- Check trùng bằng `SingleOrDefault(c => c.Name == model.Name && c.ShopId == model.ShopId)`.
- Thêm entity bằng `_unitOfWork.CategoryRepository.AddAsync(...)`.
- Bọc thao tác ghi trong `try/catch`, khi lỗi gọi `RollbackTransactionAsync()` (giống UserService).
- Trả `statusCode = 201` khi thành công, `400` khi trùng hoặc lỗi.
- Khai báo `[ProducesResponseType(...)]` cho cả 201 và 400 để Swagger mô tả rõ.

✅ Tiêu chí hoàn thành

- ☐ Gửi request thiếu `Name` → nhận lỗi validate 400 (DataAnnotation lo, không cần tự viết).
- ☐ Tạo 2 category trùng tên cùng ShopId → lần 2 báo trùng.
- ☐ Tạo thành công → DB có bản ghi mới, `Alias` sinh đúng, `Deleted = false`.
- ☐ Không hardcode chuỗi thông báo rải rác — gom vào class hằng.

🧭 Cách chạy · Kiểm tra · Chấm điểm

Chạy & kiểm tra

```
dotnet build                                          # kiểm tra biên dịch
dotnet run --project dotnet06_CybersoftMarketPlace.Api    # mở Swagger tại /swagger
```

Thang điểm gợi ý (mỗi bài 10đ)

Hạng mục	Điểm
Đúng luồng 4 tầng (Controller→Service→UnitOfWork→Repository)	4
Trả DTO đúng chuẩn `HTTPResponseData`, không lộ entity	2
Validate / xử lý trùng lặp / async đúng chuẩn	2
Đăng ký DI đầy đủ, Swagger chạy được, có `/// <summary>`	2